R: limpieza, manipulación y exploración de datos

7mo encuentro: Manipulación de datos IV

Jonatán Perren

26 de marzo de 2026

Agenda

Hasta ahora trabajamos con una sola tabla a la vez. En este encuentro aprendemos a cambiar la **forma** de una tabla y a **combinar** múltiples tablas en un único dataset listo para analizar.

¿Qué vemos hoy?

- **Reshaping:** `pivot_longer()` y `pivot_wider()` para transformar la estructura de una tabla sin perder información
- **Joins:** `left_join()`, `inner_join()`, `right_join()`, `full_join()`, `semi_join()` y `anti_join()` para combinar tablas por una clave común
- **Apilar tablas:** `bind_rows()` y `bind_cols()`

`tidyr` provee las funciones de reshaping; `dplyr` las de joins. Ambos vienen incluidos en `tidyverse`.

Datos ordenados (tidy data)

Como ya hemos visto, dataset está **ordenado** cuando sigue tres reglas: cada variable ocupa una columna, cada observación una fila, y cada unidad observacional forma una tabla separada.

El problema es que la mayoría de los datos llegan en formato **ancho**: una columna por año, por período o por categoría. Esto es cómodo para leer a simple vista pero dificulta el análisis con `dplyr` y la visualización con `ggplot2`.

`tidyr` viene incluido en `tidyverse`. Sus dos funciones centrales son `pivot_longer()` (de ancho a largo) y `pivot_wider()` (de largo a ancho). Podés encontrar el machete de `tidyr` **acá**.

`pivot_longer()`: de formato ancho a largo

Los datos del Banco Mundial, INDEC y OCDE suelen llegar con una columna por año. Para analizarlos necesitamos una columna para el año y otra para el valor.

```r
pbi <- tibble(
  pais   = c("Argentina", "Brasil", "Chile", "Uruguay"),
  `2021` = c(487, 1649, 317, 61),
  `2022` = c(633, 1920, 301, 73),
  `2023` = c(622, 2174, 345, 78)
)

pbi |> pivot_longer(
  cols       = `2021`:`2023`,  # columnas que se pivotan
  names_to   = "anio",         # nombre de la nueva col. de categorías
  values_to  = "pbi_mmd"       # nombre de la nueva col. de valores
)
```

```
> pbi |> pivot_longer(cols = `2021`:`2023`, names_to = "anio", values_to = "pbi_mmd")
# A tibble: 12 x 3
   pais      anio  pbi_mmd
   <chr>     <chr>   <dbl>
1  Argentina 2021      487
2  Argentina 2022      633
3  Argentina 2023      622
4  Brasil    2021     1649
5  Brasil    2022     1920
 ...
```

pivot_longer(): opciones avanzadas

Convertir el tipo del nombre y descartar NAs

Por defecto, los nombres de columna se convierten en texto. Con `names_transform` los convertimos directamente a entero; con `values_drop_na` descartamos las filas sin dato.

```
pbi ⊳ pivot_longer(
  cols            = 2021:2023,
  names_to        = "anio",
  names_transform = list(anio = as.integer), # "2021" -> 2021L
  values_drop_na  = TRUE
)
```

Cuando los nombres tienen prefijo común

Si las columnas se llaman `"export_2021"`, `"export_2022"`, etc., `names_prefix` quita el prefijo automáticamente.

```
datos ⊳ pivot_longer(
  cols         = starts_with("export_"),
  names_to     = "anio",
  names_prefix = "export_",
  values_to    = "exportaciones"
)
```

Para seleccionar columnas también podés usar `where(is.numeric)`, `ends_with()`, `matches()` y otros helpers de `dplyr`.

`pivot_wider()`: de formato largo a ancho

Operación inversa: los valores únicos de una columna pasan a ser los nuevos nombres de columna. Útil para comparar indicadores lado a lado, exportar a Excel o alimentar ciertos modelos.

```r
macro <- tibble(
  anio      = c(2022, 2022, 2023, 2023),
  indicador = c("inflacion", "desempleo", "inflacion", "desempleo"),
  valor     = c(94.8, 6.8, 211.4, 5.7)
)

macro |> pivot_wider(
  names_from  = indicador,  # valores de esta col. -> nombres de col.
  values_from = valor       # valores de esta col. -> celdas
)
```

```
> macro |> pivot_wider(names_from = indicador, values_from = valor)
# A tibble: 2 x 3
   anio inflacion desempleo
  <dbl>     <dbl>     <dbl>
1  2022      94.8       6.8
2  2023     211.4       5.7
```

Si hay combinaciones sin dato en los datos originales, se rellenan con `NA`. Usá `values_fill = 0` para reemplazar esos `NA` por cero automáticamente.

Datos relacionales: múltiples tablas

En la práctica, la información rara vez viene en una sola tabla. Lo normal es tener varias tablas vinculadas por una **clave común** (fijate sino en **dataset de IMDB**). Esa arquitectura se llama **base de datos relacional**.

Ejemplos típicos en economía:

- Una tabla de **países** con id, nombre y región, y otra de **indicadores** con id_país, año, PBI e inflación
- Una tabla de **empresas** con id y sector, y otra de **empleados** con id_empresa, nombre y sueldo
- Una tabla de **productos** con id y categoría, y otra de **ventas** con id_producto, fecha y monto

Un **join** combina filas de dos tablas basándose en los valores de una o más columnas clave. En `dplyr`, la familia `*_join()` implementa todos los tipos del álgebra relacional.

Hay que distinguir entre **mutating joins** (agregan columnas de la segunda tabla: `left_join()`, `inner_join()`, `right_join()`, `full_join()`) y **filtering joins** (filtran filas sin agregar columnas: `semi_join()`, `anti_join()`).

Tipos de joins

Trabajaremos con estas dos tablas. **`paises`** tiene cuatro entradas; **`datos`** tiene indicadores solo para tres de ellas, más un **`id`** que no tiene país.

```r
paises <- tibble(id = 1:4,
  pais   = c("Argentina", "Brasil", "Chile", "Uruguay"),
  region = c("Cono Sur", "Sudamérica", "Cono Sur", "Cono Sur"))

datos <- tibble(id = c(1, 2, 3, 5),
  pbi_usd   = c(633, 1920, 301, 73),
  inflacion = c(211.4, 4.6, 12.8, 8.3))
```

Función	Conserva	Nota
`left_join()`	todas las filas de la izquierda	rellena con `NA` donde no hay match
`inner_join()`	solo las filas con match en ambas	descarta toda fila sin coincidencia
`right_join()`	todas las filas de la derecha	simétrico al `left_join()`
`full_join()`	todas las filas de ambas tablas	máxima conservación, rellena con `NA`
`semi_join()`	filas de la izquierda con match	no agrega columnas
`anti_join()`	filas de la izquierda sin match	diagnóstico de pérdida de datos

`left_join()`: el join más frecuente

Conserva **todas** las filas de la tabla izquierda y agrega las columnas de la derecha donde haya coincidencia. Si no hay match, rellena con `NA`.

```
paises ⊳ left_join(datos, by = "id")
```

```
> paises ⊳ left_join(datos, by = "id")
# A tibble: 4 x 5
     id pais      region     pbi_usd inflacion
  <dbl> <chr>     <chr>        <dbl>     <dbl>
1     1 Argentina Cono Sur       633     211.4
2     2 Brasil    Sudamérica    1920       4.6
3     3 Chile     Cono Sur       301      12.8
4     4 Uruguay   Cono Sur        NA        NA
```

Uruguay queda con `NA` porque su `id` (4) no existe en `datos`. El `id = 5` de `datos` se descarta porque no tiene país correspondiente.

`left_join()` es el join por defecto en análisis: preserva la tabla base completa y enriquecés con la información disponible.

inner_join() y full_join()

inner_join(): solo el cruce

Conserva únicamente las filas que tienen coincidencia en **ambas** tablas. Toda fila sin match desaparece.

```
paises ⊳ inner_join(datos, by = "id")
# -> 3 filas: Argentina, Brasil, Chile. Uruguay e id=5 desaparecen.
```

full_join(): la unión completa

Conserva todas las filas de ambas tablas. Las combinaciones sin match se rellenan con NA.

```
paises ⊳ full_join(datos, by = "id")
```

```
> paises ⊳ full_join(datos, by = "id")
# A tibble: 5 x 5
     id pais      region     pbi_usd inflacion
  <dbl> <chr>     <chr>        <dbl>     <dbl>
1     1 Argentina Cono Sur       633     211.4
2     2 Brasil    Sudamérica    1920       4.6
3     3 Chile     Cono Sur       301      12.8
4     4 Uruguay   Cono Sur        NA        NA
5     5 NA        NA              73       8.3
```

`semi_join()` y `anti_join()`: joins de filtrado

Estos joins **no agregan columnas**: solo filtran filas de la tabla izquierda según si hay o no coincidencia en la derecha.

`semi_join()`: filtrar por coincidencia

Conserva las filas de la izquierda que tienen match en la derecha, sin agregar columnas.

```
# ¿Qué países tienen datos económicos?
paises ▷ semi_join(datos, by = "id")
# -> Argentina, Brasil, Chile  (Uruguay no aparece)
```

`anti_join()`: filtrar por ausencia

Conserva las filas de la izquierda que **no tienen** match en la derecha.

```
# ¿Qué países no tienen datos?
paises ▷ anti_join(datos, by = "id")
# -> Uruguay (id=4 no está en datos)
```

Antes de hacer un `left_join()` importante, usá `anti_join()` para ver exactamente qué filas quedarán con `NA`. Es mucho mejor saberlo de antemano que descubrirlo en el resumen final.

Especificar la clave del join

El argumento by controla qué columnas se usan como clave de unión.

Columnas con el mismo nombre en ambas tablas

```r
# join por una columna
paises ▷ left_join(datos, by = "id")

# join por múltiples columnas
ventas ▷ left_join(metas, by = c("region", "producto"))
```

Columnas con distinto nombre

```r
# "id" en paises se llama "cod_pais" en la otra tabla
paises ▷ left_join(otra, by = c("id" = "cod_pais"))

# sintaxis moderna con join_by() (más legible)
paises ▷ left_join(otra, by = join_by(id == cod_pais))
```

Si omitís by, dplyr usará todas las columnas con nombres en común. Esto puede generar joins incorrectos de forma silenciosa. Siempre especificá la clave explícitamente.

`bind_rows()` y `bind_cols()`: apilar tablas

`bind_rows()`: apilar filas

Une tablas con la misma estructura colocando una debajo de la otra. Si una columna no existe en algún tibble, la rellena con `NA`.

```r
sem1 <- tibble(trimestre = "S1", producto = c("Soja", "Maíz"),  monto = c(1200, 980))
sem2 <- tibble(trimestre = "S2", producto = c("Soja", "Trigo"), monto = c(1350, 520))

bind_rows(sem1, sem2)

# también acepta una lista: útil al leer varios archivos
bind_rows(list(sem1, sem2))
```

`bind_cols()`: apilar columnas

Une tablas con la misma cantidad de filas colocando una al lado de la otra. No verifica ninguna clave: simplemente pega por posición.

```r
identificadores <- tibble(id = 1:3, nombre = c("Ana", "Beto", "Carlos"))
mediciones      <- tibble(altura = c(165, 180, 172), peso = c(60, 85, 70))

bind_cols(identificadores, mediciones)
```

`bind_cols()` no verifica que las filas correspondan: si los tibbles no están en el mismo orden, los datos quedan mezclados sin ningún aviso. Preferí un join cuando haya una clave disponible.

Ejemplo integrador

Partimos de un dataset de PBI en formato ancho (como llega del Banco Mundial), lo combinamos con una tabla de metadatos de países, y producimos un resumen regional.

```r
pbi_ancho <- tibble(
  codigo = c("ARG", "BRA", "CHL", "URY", "PRY"),
  `2021` = c(487, 1649, 317,  61,  38),
  `2022` = c(633, 1920, 301,  73,  41),
  `2023` = c(622, 2174, 345,  78,  43)
)

metadatos <- tibble(
  codigo = c("ARG", "BRA", "CHL", "URY"),
  pais   = c("Argentina", "Brasil", "Chile", "Uruguay"),
  region = c("Cono Sur", "Sudamérica", "Cono Sur", "Cono Sur")
)

pbi_ancho %>%
  pivot_longer(
    cols      = 2021:2023,
    names_to  = "anio",
    values_to = "pbi"
  ) %>%
  left_join(metadatos, by = "codigo") %>
  group_by(region, anio) %>
  summarise(pbi_total = sum(pbi, na.rm = TRUE), .groups = "drop") %>
  arrange(region, anio)
```

```
> pbi_ancho ⊳ pivot_longer( … ) ⊳ left_join(metadatos, by = "codigo") ⊳ …
# A tibble: 9 x 3
  region     anio  pbi_total
  <chr>      <chr>     <dbl>
1 Cono Sur   2021        865
2 Cono Sur   2022       1007
3 Cono Sur   2023       1045
4 Sudamérica 2021       1649
5 Sudamérica 2022       1920
6 Sudamérica 2023       2174
7 NA         2021         38
8 NA         2022         41
9 NA         2023         43
```

Paraguay (`"PRY"`) no está en `metadatos`, así que su `region` queda como `NA` tras el `left_join()`. Un `anti_join(pbi_ancho, metadatos, by = "codigo")` antes del pipeline nos habría advertido esto.

Ejercicios

1. Tomá el dataset `pbi` del ejemplo y convertilo a formato largo con `anio` como entero. Verificá los tipos de columna del resultado.
2. Creá un tibble con 3 países y 4 años de datos económicos en formato ancho (una columna por año). Convertilo a formato largo.
3. Partiendo del resultado del ejercicio anterior, volvé al formato original.
4. En el dataset `relig_income` (incluido en `tidyr`), todas las columnas excepto `religion` representan rangos de ingresos. Convertilo a formato largo. ¿Cuántas filas tiene el resultado?
5. Creá un tibble de exportaciones con columnas `producto`, `trim_1`, `trim_2`, `trim_3` y `trim_4`. Convertilo a formato largo de forma que el número de trimestre aparezca limpio, sin el prefijo.
6. Creá el dataset `macro` del ejemplo pero agregale una segunda fila con el mismo año e indicador. ¿Qué hace `pivot_wider()` con esa duplicación?
7. En el dataset `us_rent_income` (incluido en `tidyr`), cada variable (`income` y `rent`) aparece como una fila. Convertilo a formato ancho para que cada variable sea una columna.

8. Creá `paises` y `datos` del ejemplo. Combiná las tablas preservando todos los países. ¿Cuál queda con `NA` y por qué?
9. Sobre las mismas tablas, conservá solo los países que tienen información en ambas tablas.
10. Encontrá qué `ids` de `datos` no tienen un país correspondiente en `paises`.
11. Creá dos tibbles: `alumnos` con `legajo` y `nombre`, y `notas` con `legajo` y `nota`. No todos los alumnos tienen nota. Realizá los cuatro tipos de mutating join y describí qué devuelve cada uno.
12. El paquete `dplyr` incluye `band_members` y `band_instruments`. Explorálos con `left_join()`, `inner_join()` y `full_join()`.
13. Creá dos tibbles de exportaciones de distintos semestres del año con la misma estructura. Apilá los dos y verificá que el resultado tiene la cantidad correcta de filas.
14. Agregá una columna `descuento` solo al tibble del segundo semestre del ejercicio anterior. Apilá los dos nuevamente. ¿Cómo maneja `bind_rows()` la columna que solo existe en uno de los tibbles?

15. Creá un dataset de empleo en formato largo (provincia, sector, trimestre, tasa). Convertilo a formato ancho con una columna por trimestre.
16. Partiendo del resultado anterior en formato ancho, volvé al largo y verificá que recuperás exactamente los datos originales.
17. Creá un tibble con 6 provincias argentinas y sus PBI para 2020, 2021, 2022 y 2023 en formato ancho. Convertilo a largo y calculá el crecimiento porcentual promedio anual de cada provincia.
18. Creá un tibble de precios de 3 productos en 4 supermercados en formato ancho (una columna por supermercado). Convertilo a largo y encontrá el supermercado más barato para cada producto.
19. Creá dos tibbles de exportaciones de distintos años con las mismas columnas. Apilá los dos y calculá el total exportado por producto entre ambos años.
20. Tenés dos tablas: `empresas` (id_empresa, nombre, cantidad_empleados) y `contratos` (id_empresa, monto). Encontrá los contratos firmados por empresas con más de 50 empleados, sin modificar la tabla de contratos.
21. Tenés `clientes` (id_cliente, nombre, ciudad) y `compras` (id_cliente, producto, monto). Calculá el gasto total por ciudad, considerando solo las ciudades donde al menos un cliente realizó una compra.

Ejercicios integradores

22. Tenés un dataset de PBI de 5 países en formato ancho (una columna por año) y una tabla separada con la región de cada país. Transformá el PBI a largo, uní con los metadatos, y calculá el PBI total por región y año. ¿Qué país no tiene región asignada?
23. Creá un dataset de ventas mensuales de 5 productos durante 2023 en formato largo. Convertilo a ancho, calculá el total anual por producto como columna adicional y ordená de mayor a menor.
24. Tenés dos tablas: `salarios` (legajo, area, sueldo_base) y `presentismo` (legajo, dias_trabajados). No todos los empleados tienen registro de presentismo. Calculá el sueldo final como `sueldo_base * dias_trabajados / 22` para quienes tienen registro, y el sueldo base sin modificación para el resto.
25. Creá un dataset de IPC en formato ancho con categorías como columnas y meses como filas. Convertilo a largo, calculá la variación mensual por categoría y clasificá cada mes según la intensidad de la variación. Resumí la cantidad de meses en cada categoría por producto.
26. Tenés tres tibbles de exportaciones de los tres primeros trimestres de 2023 con la misma estructura. Apilá los tres, unilos con una tabla de precios por producto, calculá el ingreso estimado y resumí por trimestre y producto.

Ejercicios integradores

27. Creá un dataset de empleados con columnas `nombre`, `ventas_q1`, `ventas_q2`, `ventas_q3` y `ventas_q4`. Convertilo a largo, identificá el trimestre de mayor venta por empleado, uní con una tabla de comisiones (trimestre, porcentaje) y calculá el bono anual de cada empleado.
28. Creá un dataset de inflación mensual de 4 países en formato ancho (una columna por mes). Convertilo a largo, calculá la inflación acumulada del año para cada país, clasificá cada mes como «alta» (8 %), «media» o «baja», y contá cuántos meses cayó en cada categoría por país.
29. Tenés dos tablas: `proyectos` (id, nombre, fecha_inicio, fecha_fin_prevista) y `avances` (id_proyecto, fecha_reporte, porcentaje_completado). Calculá los días transcurridos desde el inicio de cada proyecto, el avance más reciente reportado y si el proyecto está en tiempo o retrasado.
30. Tenés una encuesta con columnas `id`, `p_01` a `p_08` con respuestas del 1 al 5. Convertí las preguntas a formato largo, clasificá las respuestas en «en desacuerdo» (1–2), «neutral» (3) y «de acuerdo» (4–5), calculá el porcentaje de cada categoría por pregunta y volvé a formato ancho para reportar.
31. En `mtcars`, convertí los nombres de fila a una columna `modelo`. Creá dos tablas separadas: `especificaciones` (modelo, cyl, hp, wt) y `rendimiento` (modelo, mpg, qsec). Unilas de vuelta con un join y clasificá cada auto según su relación potencia/peso en tres grupos.